General Documentation report for System:

Table of Contents

14.Appendices

14.1. Glossary

14.2. Acknowledgments

14.3. Version History

1. Introduction

1.1. The purpose of this document is to provide a comprehensive, high-level overview of CirceSoft system. This report details the system's structure, primary functions, user interface components, and operational flow. It serves as the foundational technical document for the system that facilitates the command, control, and monitoring of the CirceBot platform, a lightweight semi-autonomous robot used for laying communication wires in simulated contested environments.

1.2. The scope of this document is limited to the CirceSoft web-based application, including:

- **User Interface** components for visualization (path, range, status panes, header)
- **Operator Controls** for path management (load, start, stop, waypoints, target destination)
- **Pathing** functionality, in which the user's data (waypoints, collisions, grid map) is input for a-star algorithm
- **Communication** mechanisms by which CirceSoft sends instructions to, and receives status updates from, the CirceBot

1.3. Audience

- **DEVCOM CirceSoft Project Developers**
  Any Software or Computer Engineering team which takes over the development, maintenance, or integration of the CirceSoft application and its interface with CireBot

- **System Integrators**
  Course instructors or students responsible for deploying CirceSoft and integrating new hardware into the system
- **CirceSoft Project Stakeholders**
  Individuals who need a concise technical understanding of the system's capabilities and architecture

1.4. Document Conventions

Convention	Formatting	Description
System/Component Names	**Bold Text**	Used for the names of the application, main robot, or major architectural components.
User Interface Elements	*Italic Text*	References specific buttons, visual panes, text labels, or features on the screen.
Technical Data/Code	`Monospace Text`	Used for API endpoints, file paths, variables, specific scripts, or code segments.
Notes and Warnings	> Blockquote	Highlights critical information, constraints, security warnings, or important prerequisites.

2. System Overview

2.1. System Architecture

The CirceSoft software and UI consists of means to allow a warfighter to control a semi-autonomous robot in a contested environment, by defining specific waypoints that the bot must reach and monitoring real-time

location and data of the bot. The overall system architecture will be defined by two perspectives: a user and developer perspective.

User perspective:

CirceSoft is a web-based control interface that allows planning and monitoring robot navigation. The system is currently being used via a simulation of real-world hardware data. For the user, the system consists of two main parts:

A browser-based interface where you draw paths, place obstacles, and control the robot

- Within the current system, the user has the option to define “obstacles” that represent places on the battlefield that the bot should not interact with.
- The user is also able to plot waypoints, or places on the map that the bot needs to interact with and arrive along its path.
- After both pieces of information are plotted, the system can calculate the shortest possible distance that the bot must take along that path, while also avoiding the defined obstacles.

A real-time connection to the robot that updates position, heading, and progress as the robot moves

- After the correct path is defined, the user can start the bot’s progress (via the simulation”) while also monitoring any real-world data that might come from the bot.
- The user can stop the bot’s progress along the path and start it again.
- After the path is completed, the user will be displaying a message and must clear the data to begin another run.

Developer Perspective:

The Circesoft system contains four main pieces that must simultaneously be ran/compiled through a terminal for each piece to function together:

1. Backend – The backend system contains all API endpoints and WebSocket data that a piece of the system uses to communicate with other pieces. The backend is written in Python.
2. Frontend – The user interface of the application, written in ReactJS. Contains all interactable elements and visual data for the user.
3. A* Pathfinding Algorithm – The algorithm that does computation to find the shortest possible path for user specifications.
4. Simulation – Program to simulate the bot's progress along the path, sends data in real time via a WebSocket to be displayed to the user.

2.2. Key Components

2.2.1. UI & Grid Map

The user interface (UI) is a React-based application that sends commands and retrieves results for the user. It is further detailed in **Section 4.2. User Interface.**

The grid map is a UI feature that the operator primarily interacts with to modify and set obstacles and path waypoints for CIRCEBot. API calls made by the front-end fetch and modified the obstacles.json file stored in the back end whenever the user modifies the obstacles on the gridmap. The user can drag or press the red X on each waypoint to move/delete waypoints. Note that restrictions prevent the operator from creating a path longer than the operating limit of CIRCEBot (300 feet). When the user presses "Start Bot", the path is sent to the back end via an API call.

While CIRCEBot is traversing along the path, the gridmap is unable to be modified. The state of the bot updates on the gridmap according to how much of the path it has traversed. For example, if the path is 200 feet long and completion percentage is 50%, the bot icon will appear 50% of the way on the path.

2.2.2. Simulation

The simulation is designed to be a temporary software-based test bed to stand in for the physical robot. The simulator is written in Python and allows us to simulate passing signals to and from the robot and determine edge cases.

2.2.3. A* Pathfinding Algorithm

The pathfinding algorithm is a best-cost algorithm that is neither depth first nor breadth first search. In the context of CIRCESoft, A* finds the lowest cost path with respect to the cable length while, at the same time, maintaining the shortest path possible.

2.2.4. Back End

The back end of CIRCESoft serves as a sort of nexus for all the signals and data being passed to and from CIRCEBot. Each endpoint has its own unique functionality.

GET /health

Returns “ok” to confirm that the server is running. Used for uptime or monitoring checks.

GET /current-values

Returns the robot’s current telemetry from `current_values.json`. Includes movement status, battery, and cable data.

PUT /current-values

Overwrites `current_values.json` with new telemetry data provided by the backend or simulator.

GET /directions

Returns the robot control directive (START, STOP, RESET). Supports both JSON and plain-text formats.

PUT /directions

Updates `directions.json` with a new command object to control robot motion state.

GET /waypoints

Returns the list of stored waypoints used for A* path planning.

PUT /waypoints

Validates and writes a list of waypoint objects (`r`, `c`, optional label) to `waypoints.json`.

GET /grid/manifest

Provides metadata for all grid files including existence, size, and modification time.

GET /grid/image

Returns the grid map background image used by the GUI.

GET /grid/coordinates

Returns coordinate mapping data for the grid, wrapped under `"data"`.

GET /grid/obstacles

Returns obstacle positions from `obstacles.json`, wrapped under `"data"`.

PUT /grid/obstacles

Validates and stores updated obstacle coordinates.

GET /grid/path

Returns the current A* path used by the robot.

PUT /grid/path

Accepts and stores a validated list of path points that represent a complete movement path.

GET /progress

Returns how far along the robot is on its path, including percent complete and distances.

GET /grid/bundle

Creates and streams a ZIP file containing grid-related artifacts (coords, obstacles, path, image).

POST /send-astar-message

Broadcasts a backend-sent A* message to all connected `/ws/astarmessages` WebSocket clients.

WebSocket: /ws/astarmessages

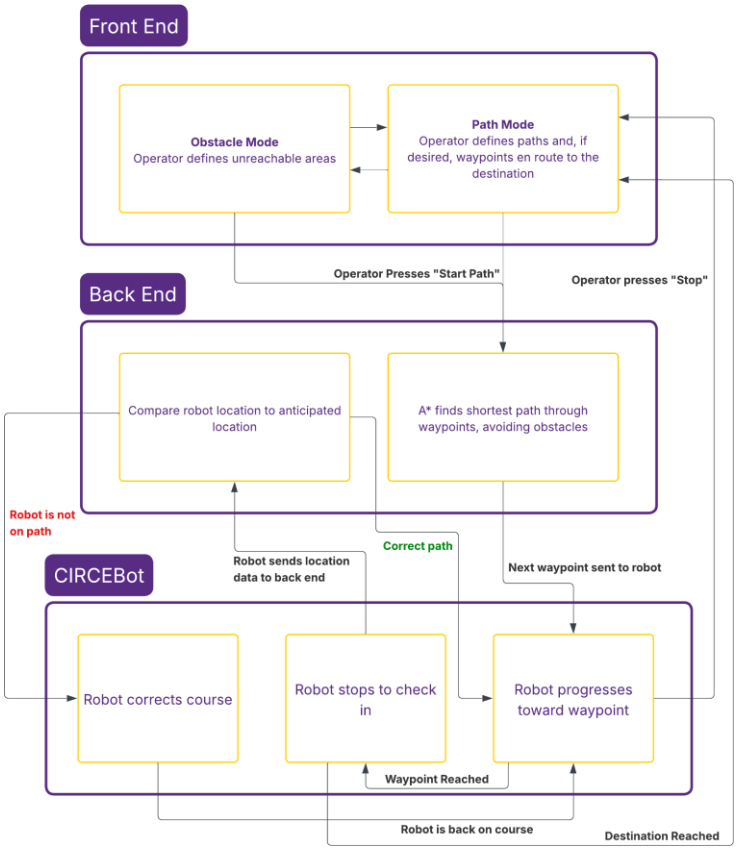
Maintains a live WebSocket connection for receiving A* notifications broadcast by backend scripts.

WebSocket: WS_PATH (main WebSocket endpoint)

Handles bi-directional real-time communication.
Receives simulator telemetry and front-end updates, and pushes updated directions, path, obstacles, and telemetry back to the UI.

List and briefly describe the key components of the system.

2.3. System Diagram



2.4. Dependencies

Several 3rd party libraries and software were used to accomplish this project.

asyncio

Used for asynchronous tasks, polling loops, WebSocket handling, and background timers. Enables non-blocking execution for the planner, simulator, and WebSocket server.

aiohttp

Used for making asynchronous HTTP requests between components (e.g., simulator → server). Handles GET/PUT operations efficiently without blocking the event loop.

json

Handles JSON encoding and decoding across all modules. Used for reading/writing state files, request bodies, responses, and WebSocket messages.

hashlib

Used to generate SHA-256 hashes of path/obstacle/waypoint data. Allows detection of changes in files or responses.

math

Provides distance calculation helpers for A* heuristics and movement cost functions.

time

Used for timestamps, cooldown timers, logging output, and ZIP file naming.

typing (List, Dict, Any, Optional, Tuple)

Provides type hints to improve readability and maintainability of the codebase.

FastAPI (FastAPI, APIRouter, WebSocket, Body, WebSocketDisconnect)

Framework powering the backend API and WebSocket services. Used for defining REST endpoints, handling WebSocket connections, and receiving structured request bodies.

fastapi.responses (PlainTextResponse, FileResponse, StreamingResponse, JSONResponse)

Allows returning different response types such as text, files, streamed ZIP bundles, and raw JSON.

uvicorn

Runs the FastAPI application server when starting `app.py`. Provides the ASGI server foundation for handling HTTP and WebSocket connections.

os

Used for file existence checks, size lookups, path building, and reading environment variables.

io

Used for building in-memory byte streams, specifically for generating ZIP archives without writing to disk.

zipfile

Used to create ZIP bundles of grid files for export. Handles compression and packaging within the `/grid/bundle` endpoint.

datetime (from simulatedSimulator)

Used to generate ISO-formatted timestamps for log entries and telemetry messages.

heapq *(inside grid_astar.py's A* implementation) *

Used to implement the A* open set priority queue. Needed to select the next node with the smallest f-cost.

utils (custom CIRCESoft module)

Contains specialized helper functions for parsing JSON, reading/writing grid files, handling telemetry, and processing WebSocket data. Not a standard library — part of your project architecture.

config (custom CIRCESoft module)

Holds file paths, constants, and configuration values used across endpoints. Enables centralized management of system-level settings.

React (Frontend Framework)

Used to build the web-based operator GUI.
Handles rendering of the grid, robot status panels, WebSocket event streams, and user interaction with waypoints/obstacles.

JavaScript / TypeScript

Core language used by the frontend for UI behavior, A* status updates, and asynchronous communication with the backend.
Likely used to manage state, emit commands, and process WebSocket messages.

WebSockets (Browser API)

Used by the frontend to maintain a live connection to the backend.
Receives streaming updates for directions, current-values, path, and obstacles.

Fetch / Axios (or similar HTTP client)

Used by the frontend to communicate with FastAPI's REST endpoints.
Handles GET/PUT requests for grid files, directions, waypoints, and more.

HTML5 Canvas / SVG (for Grid Rendering)

Used by the React UI to draw the grid, path, obstacles, and robot overlays.
Enables real-time visualization of A* movement and map updates.

CSS / Styling Libraries (e.g., Tailwind, Bootstrap, Material UI — implied)

Used to style the grid UI, control panel, and robot monitoring dashboard.
Not required in Python, but visually necessary for the frontend.

Node.js (Build + Runtime Environment for React)

Required to install packages, run the development server, and compile the production React build.
Used for Webpack/Vite tooling, bundling, and dependency management.

npm / yarn (Package Manager)

Manages frontend dependencies, React libraries, UI plugins, and build scripts.
Ensures consistent environment for building and deploying the operator interface.

3. Installation and Setup

The project utilizes a decoupled Microservice Architecture consisting of a Python Backend (FastAPI) and a JavaScript Frontend (React/Vite). Both components must be set up and run separately.

3.1. Prerequisites

The system requires two distinct runtime environments:

1. Python 3.8+: For the FastAPI Control Server, A* Pathfinder, and Simulation Driver.
2. Node.js (LTS) & npm/yarn: For the React/Vite frontend application.

Component — Environment — Key Dependencies Backend — Python 3.8+ — fastapi, uvicorn, python-dotenv, aiohttp, websockets Frontend — Node.js/npm/yarn — react, vite (inferred from .jsx files)

3.2. Installation Steps

Follow these steps to set up the environment and run both the backend and frontend components.

Step 1: Install Python Dependencies (Backend)

Install the Python libraries needed for the server, configuration, and I/O.

pip install fastapi uvicorn python-dotenv aiohttp websockets

Step 2: Create Data Directory (Backend)

The server stores all dynamic state and grid files in a local directory (default: ./data). This directory must exist before the server starts.

mkdir data

Step 3: Install Node.js Dependencies (Frontend)

Navigate to the frontend directory (if separate) and install all project dependencies.

npm install (or: yarn install)

Step 4: Run the FastAPI Control Server (Backend)

python app.py

The server runs on http://0.0.0.0:8765 by default.

Step 5: Run the Frontend Development Server

Start the React/Vite development server.

npm run dev (or: npm start)

The frontend typically opens in the browser at http://localhost:5173 or http://localhost:3000.

Step 6 (Concurrent): Run Auxiliary Components

These are required for pathfinding calculation and simulated robot status updates.

A* Pathfinder: python grid_astar.py Simulation Driver: python sim_backend_driver.py

3.3. Configuration

Configuration centers around ensuring that the frontend and backend can communicate over the correct host and port.

3.3.1. Backend Configuration (.env)

The Python backend uses a .env file to configure network and file paths.

Variable — Default Value — Description APP_PORT — 8765 — Server port. This is the target port for the frontend. CORS_ALLOW_ORIGINS — * — Must include the frontend's address (e.g., http://localhost:5173). WS_PATH — /ws — WebSocket path used for real-time updates.

3.3.2. Frontend Configuration

The React frontend must direct its API and WebSocket connections to the backend.

API Calls (REST): http://:/ Example: http://localhost:8765/waypoints

Real-time Status (WebSocket): ws://:/ws Example: ws://localhost:8765/ws

If the frontend is running on a different origin (for example, frontend on port 5173 and backend on port 8765), ensure the CORS_ALLOW_ORIGINS variable in the backend .env file is correctly set.

4. User Guide

4.1. Getting Started

First, the user should run the CIRCESoft executable file to run the software interface. The user can then connect to the bot and use the interface to send commands to CIRCEBot.

NOTE: The user should check to ensure that the aerial image has been set to the correct location. The aerial image should include the destination and starting point of CIRCEBot.

Once the software interface is open and a connection is established with CIRCEBot, it is recommended to first switch to “Edit Obstacles” and mark any obstacles that could interfere with CIRCEBot’s path on the gridmap. Once done, switch back to “Edit Path”, mark the starting position of the bot, and then click to define other waypoints/the destination on the path. Once a path has been defined, press the “Start Bot” button to begin CIRCEBot’s traversal along the path. If the bot needs to be stopped for testing, maintenance, or otherwise, press the "Stop Bot” button to stop the bot.

4.2. User Interface (Harrison)

The user interface is composed of the header bar, left panel, and right panel. The left side of the header shows the connection status of the bot—red if the connection to the bot is not established and green if the connection is established.

On the left panel, the user can find the following five buttons:

Also found on the left panel is the message window, which will display updates from the bot as well as error messages and warnings.

The right panel contains the gridmap, the start/stop bot button, the reset data button, and bot telemetry. The gridmap is the visualization of the bot and its progress, which can be edited by the operator. For testing purposes, the reset data button resets all telemetry to the initial state (300 ft of cable, 100% battery, etc.) The telemetry bar shows the distance traveled, distance remaining, length of cable onboard, battery life, cable dispenser status, and error status.

4.3. Functionality (Harrison)

To plot points and obstacles, the user can plot points on the gridmap. If the user wishes to remove the path or revert to the previous path, buttons at the top of the left panel allow them to do so.

Data sent from the bot is interpreted and displayed via the message window, telemetry bar, and grid map. For example, when the bot completes a certain percentage of their path, the grid map will update the bot’s position to that location and show a green line in the bot’s wake.

4.4. Troubleshooting

The message window provides a high degree of insight into the status of the connection and software. The most common error that the user could see is “WebSocket Disconnected”. This reflects either a lack of efficient communication between the front end and backend of the software. Another characteristic of the issue is the gridmap not loading, since it is stored in the back end. This issue can be remedied by running the “app.py” script in the back end to start the back end.

5. Admin Guide

5.1. Administrator Roles

An administrator is assumed to be a qualified technician which is able to perform operations on the back end or development side of CIRCESoft. There is no “system administrator” for CIRCESoft n a typical sense because CIRCESoft operates standalone from any network or enterprise system.

The administrator’s most common role is to change the aerial view of the contested environment, since this is an operation that cannot be done by the user in the current state of the software.

5.2. Changing the Background Image

The background image can be found in Backend/Capstone/data/current_image.jpg. The administrator can change this image to another JPEG image. Currently, the lines on the gridmap are rendered by current_values.json and thus any changes to the gridmap would involve changing the values off the JSON file. We recommend not altering the file manually and instead developing a method for doing so automatically in the future.

6. API Documentation

The CirceSoft Control Server is built using FastAPI and offers both RESTful API endpoints for configuration and status retrieval, as well as WebSocket (WS) endpoints for real-time data streaming.

The default base URL for the API is http://localhost:8765.

6.1. API Endpoints

All data is managed in JSON format, typically stored in files within the configured GRID_DIR (default: ./data).

A. Data and Status Endpoints

Endpoint: /health Method: GET Purpose: Server health check Request Body: None Response Body: "ok" (text/plain)

Endpoint: /current-values Method: GET Purpose: Retrieves the robot's current real-time status Request Body: None Response Body: JSON object containing ECI coordinates, heading, and other status fields

Endpoint: /current-values Method: PUT Purpose: Updates the robot's current status (used by the simulation driver) Request Body: JSON object such as {"x_eci": 10.5, "y_eci": 20.1, "heading_deg": 45.0, ...} Response Body: JSON status message

Endpoint: /directions Method: GET or PUT Purpose: Retrieves or sets the robot's current control instructions Request Body (PUT): JSON or plain text Response Body: JSON or text content from directions.json

B. Grid and Pathfinding Endpoints

Endpoint: /waypoints Method: GET Purpose: Retrieves the list of defined waypoints Request Body: None Response Body: JSON list of waypoint objects: [{"r": int, "c": int, "label": "START|WAYPOINT|END"}, ...]

Endpoint: /waypoints Method: PUT Purpose: Updates the list of waypoints and triggers path recalculation Request Body: Same JSON format as GET Response Body: JSON success message

Endpoint: /grid/obstacles Method: GET Purpose: Retrieves obstacle coordinates Request Body: None Response Body: JSON list: [{"r": int, "c": int}, ...]

Endpoint: /grid/obstacles Method: PUT Purpose: Updates the obstacle list and triggers path recalculation Request Body: Same JSON format as GET Response Body: JSON success message

Endpoint: /grid/path Method: GET Purpose: Retrieves the most recently computed path Request Body: None Response Body: JSON list: [{"r": int, "c": int, "cost": float}, ...]

Endpoint: /grid/path Method: PUT Purpose: Endpoint for the A* script to submit a new path Request Body: Same as GET Response Body: JSON success message

Endpoint: /send-astar-message Method: POST Purpose: Receives status/error messages from the A* script and broadcasts them via WebSocket Request Body: JSON object containing “message” Response Body: JSON status message

C. Utility Endpoint

Endpoint: /grid/bundle Method: GET Purpose: Bundles grid-related data (obstacles.json, grid_path.json, grid_coordinates.json, current_image.jpg) into a ZIP for download Request Body: None Response Body: ZIP file (application/zip)

D. WebSocket Endpoints

Main WebSocket Channel Path: /ws Data Push Interval: 25 milliseconds Functionality: Broadcasts all changes to directions, current-values, path, and obstacles; receives messages from the simulation driver

A* Messages WebSocket Channel Path: /ws/astarmessages Data Push Interval: Immediate push Functionality: Broadcasts pathfinding status/error messages received from /send-astar-message

6.2. Authentication

The CirceSoft Control Server does not implement authentication (no API keys, OAuth, JWT, etc.).

Security Model: Access control is expected to be enforced by network-layer protections (firewall, VPN, local network). CORS: Enabled, default set to "*", permitting all origins. This was intentional as the connection is only a hardwired one for a netowrk that is airgapped.

6.3. Usage Examples

1. GET robot status (/current-values)

   curl -X GET http://localhost:8765/current-values

2. PUT new waypoints (/waypoints)

   curl -X PUT http://localhost:8765/waypoints -H "Content-Type: application/json" -d '[{"r": 1, "c": 1, "label": "START"}, {"r": 15, "c": 20, "label": "WAYPOINT"}, {"r": 30, "c": 47, "label": "END"}]'

3. POST A* status message (/send-astar-message)

curl -X POST http://localhost:8765/send-astar-message -H "Content-Type: application/json" -d '{"message": "Path calculation successful: 152.34 ft"}'

6.4. Rate Limits

The REST API does not enforce rate limits.

Internal throttling:

1. WebSocket Broadcast Throttling: The /ws channel limits updates to once every 25 milliseconds.
2. A* Pathfinding Cooldown: The grid_astar.py script enforces a 2-second cooldown before recomputing the path after a change in waypoints or obstacles.

7. Data Model

7.1. Data Flow

Data flow in the CirceSoft system follows three concurrent loops, all synchronized through the Backend (FastAPI), which acts as the central data hub using the JSON files in the ./data directory.

I. Input and Path Generation (Command Cycle)

- User Interaction:
  - Frontend UI sends waypoints and collision masks (obstacles) to the Backend via REST PUT requests or WebSocket messages.
- Storage:
  - Backend writes this data directly to waypoints.json and obstacles.json.
- A-star Calculation:

    - An external A-star script detects changes in the input files. It uses these constraints and the map scaling from grid_coordinates.json to calculate the optimal path.
- Path Output:
    - The A-star script sends the detailed path to the Backend, which stores it in path.json.

II. Control Flow

- Command Polling:
    - The Simulator constantly polls the Backend's /directions endpoint, reading the current command (START or STOP) from directions.json.
- State Change:
    - The Simulator transitions its state (e.g., to the MOVEMENT PHASE) based on the read command and the availability of a path in path.json.
- Path Polling:
    - The Simulator also polls /grid/path (reading path.json) and uses a hash check to ensure it executes the latest path.
- Completion Signal:
    - Upon route completion, the Simulator sends a PUT request to the /directions endpoint, setting the command to "STOP" to signal the route end.

III. Real-Time Telemetry and Visualization (Feedback Cycle)

- Status Generation:
    - The Simulator calculates its current status (Position, Velocity, Metrics) every tick, converting grid r,c to X_ECI,Y_ECI (feet) using the Config data.
- Telemetry Write:
    - The Simulator sends the full status payload to the Backend via a PUT request to /current-values. The Backend overwrites current_values.json.

- Synchronization & Retrieval:
    - The Backend's WebSocket endpoint continually polls current_values.json and other key data files.
- Frontend Update:
    - When changes are detected, the Backend broadcasts the updated content to all connected Frontend clients (e.g., as current_values_update messages) for visualization.

7.2. Data Entities

The system data is organized into configuration, user input, calculation output, and real-time telemetry.

I. System Configuration and Environment (Static Files)

- Grid Configuration (grid_coordinates.json)
    - Role: Defines the conversion factors between abstract grid coordinates (r,c) and physical coordinates in feet (X,Y).
    - Key Attributes:
        - X_SCALE (Feet per column grid step)
        - Y_SCALE (Feet per row grid step)
        - DIAG_SCALE (Feet per diagonal step)
- Path Commands/Control State (directions.json)
    - Role: Stores the current high-level command controlling the Simulator's state.
    - Key Attributes:
        - command (Text value: "START" or "STOP")
        - (Implicit) last_updated (Timestamp of the last command write)

II. Pathing Inputs (User-Defined)

- Waypoints (waypoints.json)
    - Role: The sequence of key points (start, intermediate, end) that the A-star algorithm must connect.
    - Key Attributes:

        - r (Row coordinate, float)
        - c (Column coordinate, float)
        - label (Optional identifier/name)
- Obstacles (obstacles.json)
    - Role: Coordinates marked by the user as impassable or collision zones.
    - Key Attributes:
        - r (Row coordinate, float)
        - c (Column coordinate, float)

III. Pathing Output (A-star Result)

- A-star Path (path.json)
    - Role: The detailed, ordered sequence of coordinates defining the optimal, collision-free path.
    - Key Attributes:
        - List of Points, where each point contains:
            - r (Row coordinate, float)
            - c (Column coordinate, float)

IV. Real-Time Telemetry (Bot Status)

- Current Values (current_values.json)
    - Role: Comprehensive, single-state snapshot of the Bot's real-time physical and status data.
    - Position & Velocity (Feet)
        - X_ECI, Y_ECI (Current position)
        - Vx_ECI, Vy_ECI (Current velocity vectors)
    - Movement & State
        - isMoving (Boolean status)
        - Heading (Text label)
        - SequenceNum (Current waypoint number, 1-based)
    - Resource & Metrics
        - percentBatteryRemaining (Integer percentage)
        - cableRemaining (Float distance in feet)

        - distanceTraveled (Accumulated distance in feet)
        - distanceRemaining (Remaining path distance in feet)
    - Error Reporting
        - errorCode (Status code/value)
        - cable flags (Status flags related to the tether)

8. Deployment

8.1. Deployment Architecture

The system is best deployed as a microservice-style structure that separates the web interface, the API and control logic, and the computational work performed by the A* pathfinding system.

Key Components:

1. Reverse Proxy (such as Nginx or Traefik). This acts as the entry point for all client traffic. It handles HTTPS termination, routes REST API calls to the FastAPI backend, and manages WebSocket upgrade requests to maintain persistent real-time communication channels.
2. FastAPI Control Server (app.py). This service runs using a production ASGI server such as Gunicorn with Uvicorn workers. It is responsible for all REST endpoints (reading and updating data) and for hosting the primary WebSocket channels (/ws and /ws/astarmessages).
3. A* Pathfinding Worker (grid_astar.py). This is an independent Python process that runs continuously. It communicates with the FastAPI server through REST endpoints, retrieving grid and waypoint data and submitting computed path results and status messages.
4. React Frontend (static build files). The React codebase is built into static HTML, CSS, and JavaScript assets. These files are served directly by the reverse proxy or by a dedicated web server or CDN.
5. Data Storage (./data directory). Shared JSON data files that the worker and the FastAPI server both read and write must be stored in

a shared, persistent location. For containerized deployments, this typically means using a mounted volume.

8.2. Deployment Steps

The following steps assume a Linux server environment or a container platform such as Docker or Kubernetes.

Step 1: Configure Environment Variables

Create the required environment variables for production and set them in a .env file or via container secrets.

APP_HOST: typically set to 0.0.0.0 APP_PORT: typically set to 8765
CORS_ALLOW_ORIGINS: set to the public frontend domain
GRID_DIR: set to a persistent directory for shared JSON data files

Step 2: Build the Frontend

Install dependencies and generate production-optimized static files.

npm install npm run build

The final static files will appear in the build or dist directory.

Step 3: Deploy the Backend

Use Gunicorn together with Uvicorn workers to run the FastAPI application with multiple workers.

Example command: gunicorn app:app -k uvicorn.workers.UvicornWorker -w 4 --bind 0.0.0.0:8765

Step 4: Deploy the A* Worker

Run grid_astar.py as a managed service (using systemd, Supervisor, Docker, or Kubernetes). The service should automatically restart if it stops.

Note: sim_backend_driver.py should not be used in production. In a real deployment, it is replaced by communication with the physical robot hardware or telemetry systems.

Step 5: Configure the Reverse Proxy

The reverse proxy must:

1. Serve the static frontend files on the root path.
2. Forward API traffic to the FastAPI server at http://localhost:8765.
3. Correctly handle WebSocket upgrade headers for the /ws and /ws/astarmessages endpoints.

8.3. Scaling and Load Balancing

Different system components have different scaling strategies.

8.3.1. Scaling the FastAPI Server

The FastAPI server is stateless and can be scaled horizontally. Multiple instances can run behind a load balancer. Because the servers read and write to shared JSON data, the storage backing GRID_DIR must be shared. This can be implemented using an NFS mount or a cloud-managed shared filesystem such as AWS EFS or Google Cloud Filestore.

8.3.2. Scaling the A* Pathfinding Worker

The A* worker is designed as a single-instance process. Running multiple copies would cause conflicting reads and writes to grid_path.json and other shared files. Instead of horizontal scaling, focus should be placed on reliability, monitoring, and automatic restart mechanisms.

8.3.3. Scaling the Frontend

The frontend consists solely of static files and scales easily using a CDN such as Cloudflare or AWS CloudFront, providing fast delivery and reducing load on backend systems.

9. Security

9.1. Authentication and Authorization

CirceSoft operates in an air-gapped environment where all communication occurs over local host, so user authentication is not currently implemented. Physical access to the operator device serves as the primary access control mechanism. API endpoints such as `/grid/path` and `/current-values` do not require authentication tokens in this deployment model.

Future networked deployments should implement:

- Authentication on all API endpoints to verify request sources
- Mutual verification between CirceSoft and CirceBot to prevent entity impersonation
- Source verification on telemetry endpoints to ensure only the authenticated robot can update its status

9.2. Encryption

The system does not currently encrypt data in transit or at rest, which is acceptable given the air-gapped, localhost-only architecture. The primary encryption concern is the physical Ethernet tether between the operator device and CirceBot, which could be spliced by an attacker to intercept path data, telemetry, or command signals.

9.3. Security Best Practices

Operate CirceSoft only on hardened, isolated devices and disable unused wireless radios (Wi-Fi, Bluetooth) on CirceBot's Raspberry Pi to eliminate unnecessary attack vectors. All API inputs should be validated and rate-limited to prevent injection attacks and denial-of-service

conditions. Treat CirceBot as a potentially compromised device—never trust incoming data without validation.

Hardware security enforcement:

- 300ft maximum cable length enforced in the A* pathfinder, with server-side redundant validation recommended
- Real-time cable tracking with anomaly detection for position vs. cable usage discrepancies
- Plausibility checks on telemetry to reject physically impossible position updates

10. Monitoring and Logging

10.1. Logging Mechanisms

CirceSoft has a “message box” logging environment to provide confirmation for a user’s actions and is displayed to them in a section of the main UI. Along with this, there are various console logs and try-except logs for data and action confirmation within the development environment.

The message box provides confirmation for successful API calls for critical data adjustments and changes to data within the frontend. Some of the displayed messages are due to the following actions:

- Succeeding/Failing to connect to the WebSocket
- Plotting and exporting obstacles
- Plotting and exporting waypoints/path
- Receiving calculated paths
- Succeeding/Failing to send commands
- Bot start/stop status

For developers: If you're adding a new feature that should notify the user, you'll pass **messageBoxRef** to your component and call

messageBoxRef.current.addMessage(type, content) where **type** is a string like "**info**", "**warning**", or "**error**".

10.2. Monitoring Tools

The interface provides several real-time indicators so operators can monitor the robot's status:

Indicator	Location	Display
Connection icon	Top-left header	Whether the app is connected to the robot backend
Moving status	Header / bottom controls	Whether the robot is currently in motion
Progress bar	Below the map	How far along the robot is on its current path
Current Values panel	Bottom-right	Live telemetry: position, heading, distance traveled

10.3. Alerts and Notifications

CirceSoft uses non-intrusive visual alerts rather than popups or sounds. Here is how these notifications are displayed:

- Message Window - Errors and warnings appear here with distinct coloring, so they stand out from normal messages
- Connection icon - Changes appearance immediately if the backend connection is lost
- Unsaved changes - The interface tracks whether you have unsaved modifications to your path or obstacles, which can trigger warnings before destructive actions

What to watch for:

- Red or yellow messages in the Message Window indicate something needs attention
- If the connection icon shows disconnected status, the robot won't respond to commands until reconnected
- If progress stops unexpectedly, check the Message Window for error details

11. Maintenance and Updates

11.1. Patching and Upgrades

All patches have been applied using Git to pull new changes.

11.2. Backup and Recovery

Git and branch protections were utilized to prevent unnecessary overwrites or deletes. Branching was utilized to protect production features.

11.3. Version Control

The team had a public GitHub repository setup to maintain version control as well as all memebers had locally cloned repositories on their machines.

12. Troubleshooting

12.1. Common Issues

Common issues within the system could include:

Networking issues/API Errors – Because the application is hosted on ports in localhost, make sure the correct ports are defined and each portion of the system is running together (backend, simulation, etc.).

Common HTTP Errors

- 404 Not Found
    - The requested file doesn't exist (e.g., current_values.json, directions.json, path/obstacle files)
    - Check that the GRID_DIR and file paths in config.py are correct
    - Verify files were created before first request (some endpoints require pre-existing files)
- 400 Bad Request
    - Invalid JSON format on PUT requests
    - Missing required r and c keys in path, obstacle, or waypoint data
    - Example valid format: [{"r": 0, "c": 0}, {"r": 1, "c": 2}]
- 500 Internal Server Error
    - File write permission issues—check that the server can write to the data directory
    - JSON encoding failures—verify incoming data is serializable
    - Check server console logs for the specific exception message

WebSocket Issues

- Connection drops immediately
    - Confirm the server is running on the expected port (localhost:8765)
    - Check for CORS issues if connecting from a different origin
    - Verify WS_PATH in config matches the frontend connection URL
- Not receiving updates
    - The WebSocket polls files every 25ms—ensure files are actually changing
    - Updates only push when content differs from last sent value (hash comparison)
    - Check last_sent_* tracking variables if updates seem stuck

Incorrect simulation data – Verify the simulator is using the correct port and endpoints and verify that the backend is running correctly. For technical simulation errors, ensure commands are received correctly through the “directions” endpoint.

12.2. Debugging Techniques

For CirceSoft’s current use case, the CirceSoft System is separated into four different system components that need to be run/compiled separately in order to troubleshoot each component. These sections consist of:

1. Backend
2. Frontend
3. Receiver for A* Path Calculations
4. Simulation (this will likely be changed as hardware is introduced to the project in the future)

To run each portion of the project as a developer in localhost, the developer must be located within the CirceSoft repository. The following describes the steps to run each section in separate terminals:

Run the backend

- From the root **CirceSoft** directory, navigate to **CirceSoft/Backend/Capstone/scripts/**
- Use **./Run.sh**
- This script creates a python environment and installs all dependencies to run the backend, and then runs **app.py** within **CirceSoft/Backend/Capstone/**

Run the frontend

- From the root CirceSoft directory, navigate to **CirceSoft/Frontend/react/my-app/**
- Use **npm install**
- Use **npm run dev**

- From here, you will be given a localhost address for your frontend. Enter this into your browser, and you can see the frontend from there.

Run the A* pathing algorithm receiver

- From the root **CirceSoft** directory, navigate to **CirceSoft/AI-Pathfinding/**
- Use **python grid_astar.py** or **python3 grid_astar.py**
    - If this does not work, make sure you have the correct python version installed on your machine.

Run the Simulation

- From the root CirceSoft directory, navigate to **CirceSoft/Simulation/Simulation-Python/**
- Use **python simulatedSimulator.py** or **python3 simulatedSimulator.py**
    - If this does not work, make sure you have the correct python version installed on your machine.

13. Support and Contact

13.1. Support Channels

Eric Brown- Capstone Advisor for CSC 4615

Ben Burchfield- Capstone Advisor for CSC 4620

Elias Brady- Point of contact with DEVCOM

Joze Lindquist- Software Engineering expert with DEVCOM

Coby Smith- Leader of the Autonomous Robotics Club

Capstone Developer Group for CSC 4615-001: CirceSoft project

13.2. Contact Information

ctsmith49	931-372-3602

Eric Brown	ELBrown@tntech.edu
Ben Burchfield	931-372-3122 bburchfield@tntech.edu
Elias Brady	elias.t.brady.civ@army.mil
Joze Lindquist	joze.t.lindquist.civ@army.mil
Coby Smith	ctsmith49@tntech.edu
Celia Hough	chough42@tntech.edu
JP Ognibene	jpognibene42@tntech.edu
Harrison Simpson	ghsimpson42@tntech.edu
Graham Wheeler	rgwheeler42@tntech.edu
Gabriel Adams	gradams42@tntech.edu
Ryan Naleway	rnaleway42@tntech.edu

14. Appendices

14.1. Glossary

ACRONYMS

A* – A-star; a best-cost pathfinding algorithm that finds the shortest path between two points while avoiding obstacles

API – Application Programming Interface; a set of protocols allowing software components to communicate

ASGI – Asynchronous Server Gateway Interface; a standard interface between async Python web servers and applications

C2 – Command and Control; military term for the exercise of authority over assigned forces

CDN – Content Delivery Network; a distributed server system that delivers web content based on user location

CORS – Cross-Origin Resource Sharing; a security mechanism that allows or restricts web requests from different domains

CPS – Cyber-Physical System; a system integrating computation, networking, and physical processes

ECI – Earth-Centered Inertial; a coordinate reference frame used for positioning (X_ECI, Y_ECI, Z_ECI)

GUI – Graphical User Interface; a visual interface allowing users to interact with software

HA – High Availability; system design ensuring continuous operation with minimal downtime

HMI – Human-Machine Interface; the user interface connecting an operator to a machine or system

JSON – JavaScript Object Notation; a lightweight data format used for storing and transmitting structured data

JWT – JSON Web Token; a compact token format for securely transmitting information between parties

LiDAR – Light Detection and Ranging; a sensing method using laser light to measure distances

NFS – Network File System; a protocol allowing file access over a network

npm – Node Package Manager; the default package manager for Node.js

REST – Representational State Transfer; an architectural style for designing networked applications using HTTP methods

WS – WebSocket; a communication protocol providing full-duplex channels over a single TCP connection

VPN – Virtual Private Network; an encrypted connection over the internet between a device and a network

PROJECT-SPECIFIC TERMS

CirceSoft / CIRCESoft – Cable Installation Robot for Contested Environments Software; the web-based control interface for CirceBot

CirceBot / CIRCEBot – The semi-autonomous robot hardware controlled by CirceSoft, designed to lay up to 300ft of Ethernet cable

DEVCOM – U.S. Army Combat Capabilities Development Command; the military organization sponsoring this project

Gridmap – The interactive visual grid in the UI where operators place waypoints and obstacles

Message Window – The scrollable panel in the UI displaying system logs, warnings, and status updates

Operator Device – The hardened computer running CirceSoft that communicates with CirceBot

TECHNICAL TERMS

Air-gapped – A network security measure where a computer or network is physically isolated from unsecured networks

Broadcast – Sending a message simultaneously to all connected clients

Endpoint – A specific URL path where an API receives requests (e.g., /grid/path)

FastAPI – A modern Python web framework for building APIs with automatic documentation

Hash – A fixed-size string generated from data, used to detect changes (e.g., SHA-256)

Localhost – The default hostname referring to the current device (127.0.0.1)

Microservice – An architectural style structuring an application as a collection of loosely coupled services

Obstacle – A grid cell marked as impassable that the pathfinding algorithm must route around

Path – The calculated sequence of coordinates representing the robot's route from start to destination

Polling – Repeatedly checking a resource at regular intervals for changes

Raspberry Pi – A small single-board computer used as CirceBot's onboard controller

React – A JavaScript library for building user interfaces, used for the CirceSoft frontend

Reverse Proxy – A server that forwards client requests to backend servers (e.g., Nginx, Traefik)

Telemetry – Automated collection and transmission of data from remote sources (e.g., robot position, battery level)

Uvicorn – A lightning-fast ASGI server used to run FastAPI applications

Vite – A modern frontend build tool providing fast development server and optimized production builds

Waypoint – A defined point on the grid that the robot must reach along its path

WebSocket – A protocol enabling persistent, bidirectional communication between client and server

UNITS AND MEASUREMENTS

ft (feet) – The primary unit of distance measurement in CirceSoft

X_SCALE – Conversion factor of 7.5 feet per column grid step

Y_SCALE – Conversion factor of 5.16129 feet per row grid step

DIAG_SCALE – Conversion factor of 9.104334 feet per diagonal grid step

300ft limit – Maximum cable length CirceBot can deploy; enforced by the A* algorithm

FILE REFERENCES

current_values.json – Stores real-time robot telemetry (position, battery, cable remaining, etc.)

directions.json – Stores the current control command (START, STOP, RESET)

waypoints.json – Stores user-defined waypoints for path calculation

obstacles.json – Stores grid coordinates marked as impassable

grid_path.json – Stores the calculated A* path

grid_coordinates.json – Stores grid-to-feet conversion configuration

current_image.jpg – The aerial background image displayed on the gridmap

14.2. Acknowledgments

Eric Brown – Capstone Coordinator
Elias Brady (DevCom) - Client
Developers and team members:

Member	Role
Celia Hough	Project Coordinator

JP Ognibene	Robot Sim and A* pathing algorithm
Harrison Simpson	Frontend Developer
Graham Wheeler	Lead Frontend Developer
Gabriel Adams	Backend Developer
Ryan Naleway	Frontend Developer

14.3. Version History

Iteration 1:

Deciding project goals and communicating with the team.

Iteration 2:

Sep 19, 2025

Migrated from basic HTML and JavaScript to ReactJS.

Test WebSocket communication was introduced.

Iteration 3:

Oct 5, 2025

Built a simulation-based test environment to validate system functionality independently of external teams.

Established complete data flow between the React frontend, Fast API backend, and A* pathfinding algorithm.

Iteration 4:

Oct 26, 2025

Integrated backend connections with the frontend grid map, enabling users to retrieve and send paths and obstacles. Added enhanced functionality including waypoint removal/rearrangement and a new "Obstacle Mode" for plotting obstacles.

Developed a digital simulation testbed to test CirceSoft without physical hardware. Made major progress on the A* algorithm to interpret frontend waypoints and calculate optimal paths around obstacles.

Iteration 5:

Nov 16, 2025

MVP FINSHED

Implemented complete backend support for route importing path comparison, deviation alerts, and real-time simulation with distance and cable tracking. Enhanced the A* pathfinding to compute optimal paths using collision masks and automatically identify unreachable terrain as obstacles.

Significantly improved the gridmap with structured lines, snapping functionality, obstacle mode toggling, and interaction locking during bot operations. Established robust backend-frontend communication for transmitting errors, path rejections, completion notifications, and real-time progress updates

Iteration 6:

Dec 5, 2025

Finished product – completed

Minor Bug fixes and containerization of the product.

JP Ognibene

Graham Wheeler

Gabriwl Adams

Ryan Naleway

Celia Hough

Elias Brady

Joze Lindquist